

Chapter 6

Networking

1 Introduction

- Networking is the communication of two processes , executing on same or different machines, over the network.
- With networks, a single program can regain information stored in computers positioned anywhere in the world.
- Java.net is a package that provides a set of classes and interfaces for network programming.
- The java.net package provides support for the two common network protocols:
 - ◆ TCP: Permits reliable communication between 2 applications. It is denoted as TCP/IP.
 - ◆ UDP: It is a connection-less protocol.

2 InetAddress class

- The InetAddress class is used to encapsulate both the numerical IP address and the domain name for that address.
- InetAddress can handle both IPv4 and IPv6 addresses.
- It has no visible constructors.
 - ◆ To create an InetAddress object, you have to use one of the available factory methods.
 - ➔ static InetAddress getLocalHost()
 - ➔ static InetAddress getByName(String hostName)
 - ➔ static InetAddress[] getAllByName(String hostName) :- returns an array of IP addresses of a host
 - ➔ static InetAddress getByAddress(String hostName)
 - ◆ All of the factory methods throws UnknownHostException

2 InetAddress class - Example

```
public static void main(String[] args) throws UnknownHostException{  
    InetAddress addr1 = InetAddress.getLocalHost();  
    System.out.println(addr1);  
    InetAddress addr2 = InetAddress.getByName("www.google.com");  
    System.out.println(addr2);  
    InetAddress[] addr3 = InetAddress.getAllByName("www.yahoo.com");  
    for (InetAddress addr : addr3) {  
        System.out.println(addr);  
    }  
}
```

3 TCP/IP Sockets

- Are used to implement reliable, bidirectional, persistent, point-to-point, stream-based connections between hosts on the Internet.
- A socket can be used to connect Java's I/O system to other programs
- There are two kinds of TCP/IP sockets in Java.
 - ◆ The ServerSocket class is designed to be a "listener".
 - ➔ waits for clients to connect before doing anything.
 - ➔ It is for servers.
 - ◆ The Socket class is designed to connect to server sockets and initiate protocol exchanges.
 - ➔ It is for clients

3.1 The Socket class

- The creation of a Socket object implicitly establishes a connection between the client and a server.
- Has two constructors:
 - ◆ Socket(String hostName, int port) throws UnknownHostException, IOException
 - ➔ Creates a socket connected to the named host and port.
 - ◆ Socket(InetAddress ipAddress, int port) throws IOException
 - ➔ Creates a socket using a preexisting InetAddress object and a port.
- It defines several methods. Some of its methods are:
 - ◆ getInputStream() throws IOException
 - ➔ Returns the InputStream associated with the invoking socket.

3.1 The Socket class

- ◆ `getOutputStream( )` throws `IOException`
 - Returns the `OutputStream` associated with the invoking socket.
- ◆ `InetAddress getAddress( )` throws `IOException`
 - Returns the `InetAddress` associated with the `Socket` object.
- ◆ `int getPort( )` Returns the remote port to which the invoking `Socket` object is connected.
- ◆ `int getLocalPort( )` Returns the local port to which the invoking `Socket` object is bound
- ◆ Its `close( )` method is used to close a socket.
 - Closing a socket also closes the I/O streams associated with the socket.
- ◆ It implements `AutoCloseable`,
 - you can use a try-with-resources block to manage a socket.

3.1 The Socket class - Example

```
1 public static void main(String [] args) throws Exception{
2     InetAddress addr = InetAddress.getLocalHost();
3     Socket s = new Socket(addr, 8888); //create client socket
4     InputStream is = s.getInputStream(); //get associated input stream
5     OutputStream os = s.getOutputStream(); //get output stream
6     DataInputStream dis = new DataInputStream(is);
7     DataOutputStream dos = new DataOutputStream(os);
8     dos.writeUTF("Hello server, I need your help"); //send message
9     System.out.println(dis.readUTF()); //receive and print message
10    s.close(); //close the socket
11 }
```


3.2 The ServerSocket class

- The creation of a ServerSocket object create a server socket and bind it to a specific port number
- Has the following constructors:
 - ◆ ServerSocket(int port):
 - ➔ creates a server socket that is bound to the specified port number.
 - ◆ ServerSocket(int port, int backlog):
 - ➔ backlog specifies the maximum number of queued connections.
 - ◆ ServerSocket(int port, int backlog, InetAddress bindAddr):
 - ➔ Binds the server socket to the specified local IP address.

3.2 The ServerSocket class

- Its `accept()` method creates a `Socket` class object and start listening for incoming client requests.
 - ◆ It blocks the current thread until a connection is made.
 - ◆ And the connection is represented by the returned `Socket` object.
 - ◆ Use the sockets input and output streams
- Once a `Socket` object is returned, use
 - ◆ The `InputStream` associated to the socket to read data from the client
 - ◆ The `OutputStream` associated to the socket to send data to the client,
- To stop the server, call the `ServerSocket`'s `close()` method.
 - ◆ Then all currently connected clients will be disconnected.

3.2 The ServerSocket class

- The following server waits for a client to connect, serve it and close.

```
ServerSocket serverSocket = new ServerSocket(port);  
Socket socket = serverSocket.accept();  
// read data from the client  
// send data to the client
```

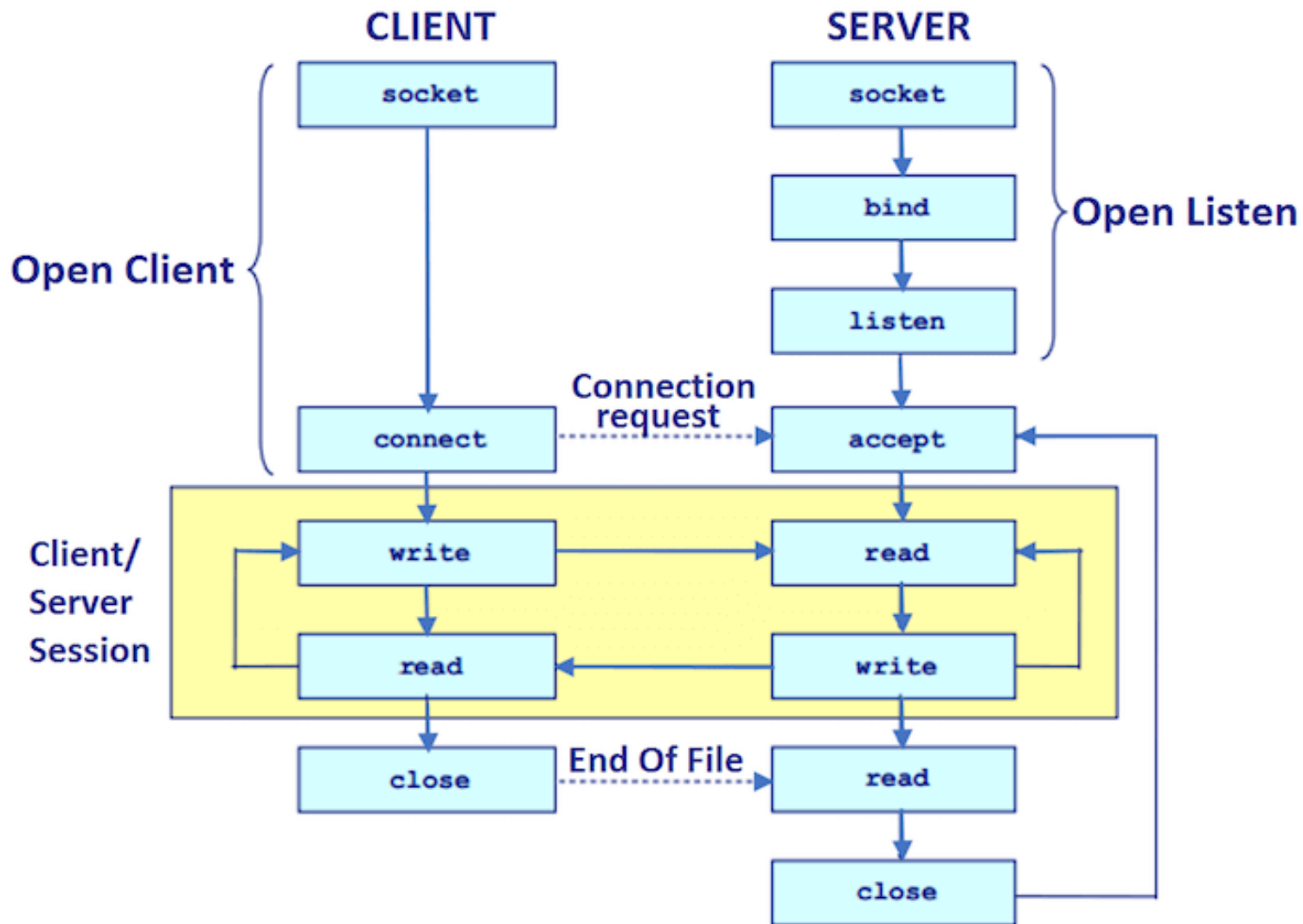
- The while(true) loop allows the server to run forever. However, the server serves only one client at a time.

```
ServerSocket serverSocket = new ServerSocket(port);  
while (true) {  
    Socket socket = serverSocket.accept();  
    // read data from the client  
    // send data to the client  
}
```

3.2 The ServerSocket class

- Threads can be used to make the server more responsive.
 - ◆ Each client socket is handled by a new thread.
 - ◆ The server's main thread is only responsible for listening and accepting new connections.

```
while (true) {  
    Socket socket = serverSocket.accept();  
    // create a new thread to handle client socket  
}
```



3.2 The ServerSocket class - Example

```
1 public static void main(String args[]) throws Exception {  
2     ServerSocket ss = new ServerSocket(8888);//create server socket  
3     System.out.println("server is waiting ");  
4     Socket s = ss.accept();//wait for a client connection  
5     InputStream is = s.getInputStream();//get associated input Stream  
6     OutputStream os = s.getOutputStream();//get output stream  
7     DataInputStream dis = new DataInputStream(is);  
8     DataOutputStream dos = new DataOutputStream(os);  
9     System.out.println(dis.readUTF());//print the received message  
10    dos.writeUTF("Hello dear client, u r welcome");//send message  
11    ss.close();  
12 }
```

4. Datagrams

- Datagrams are bundles of information passed between machines.
- Java implements datagrams on top of the UDP protocol by using two classes:
- The DatagramPacket class is used to create the data container.
- The DatagramSocket class is used to send or receive DatagramPackets.

4.1 The DatagramSocket Class

- Has the following constructors:
 - ◆ DatagramSocket() throws SocketException
 - ➔ creates a DatagramSocket bound to any unused port on the local computer.
 - ◆ DatagramSocket(int port) throws SocketException
 - ➔ creates a DatagramSocket bound to the port specified by port.
 - ◆ DatagramSocket(int port, InetAddress ipAddr) throws SocketException
 - ➔ Creates a DatagramSocket bound to the specified port and InetAddress.
 - ◆ DatagramSocket(SocketAddress address) throws SocketException
 - ➔ Constructs a DatagramSocket bound to the specified InetSocketAddress.
 - ➔ InetSocketAddress encapsulates an IP address with a port number.

4.1 The DatagramSocket Class

- Has many methods. Two of its most important methods are `send( )` and `receive( )`
 - ◆ `void send(DatagramPacket packet)` throws `IOException`
 - ➔ Sends a packet to the port specified by `packet`.
 - ◆ `void receive(DatagramPacket packet)` throws `IOException`
 - ➔ Waits for a packet to be received and returns the result.
 - ◆ Its `close( )` method, closes the socket.
 - ◆ It implements `AutoCloseable`
 - ➔ can be managed by a try-with-resources block.

4.2 The DatagramPacket Class

- Defines several constructors. Four of them are:
 - ◆ DatagramPacket(byte[] data, int size)
 - ➔ Specifies a buffer that will receive data and the size of a packet.
 - ➔ It is used for receiving data over a DatagramSocket.
 - ◆ DatagramPacket(byte[] data, int offset, int size)
 - ➔ Specify an offset into the buffer at which data will be stored.
 - ◆ DatagramPacket(byte[] data, int size, InetAddress ipAddress, int port)
 - ➔ specifies a target address and port, which determine where the data in the packet will be sent.
 - ◆ DatagramPacket(byte[] data, int offset, int size, InetAddress ipAddress, int port)
 - ➔ Transmits packets beginning at the specified offset into the data.

4.2 The DatagramPacket Class

- Defines several methods.
 - ◆ InetAddress getAddress() :-
 - ➔ Returns the address of the source or destination of a Datagram
 - ◆ byte[] getData() :-
 - ➔ Returns the byte array of data contained in the datagram.
 - ◆ int getLength():-
 - ➔ Returns the length of the valid data contained in the Datagram
 - ◆ int getOffset():-
 - ➔ Returns the starting index of the data.
 - ◆ int getPort()
 - ➔ Returns the port number.

4.3 A Datagram - Example

```
1 public class MyDatagramClient {  
2     public static void main(String args[]) throws Exception{  
3         DatagramSocket dgs = new DatagramSocket();  
4         String str = "Hello server, I need your help";  
5         byte [] b = str.getBytes();  
6         InetAddress addr = InetAddress.getLocalHost();  
7         DatagramPacket dp = new DatagramPacket(b, b.length, addr, 8888);  
8         dgs.send(dp); //send packet to server  
9         System.out.println("client is waiting for response");  
10        dgs.receive(dp); //receive packet from server  
11        System.out.println(new String(dp.getData(), 0, dp.getLength()));  
12    }}
```

4.3 A Datagram - Example

```
1 public class MyDatagramServer {
2     public static void main(String args[]) throws Exception {
3         DatagramSocket dgs = new DatagramSocket(8888); //bing port to socket
4         byte [] buf = new byte[1024];
5         DatagramPacket dgp = new DatagramPacket(buf, buf.length);
6         System.out.println("server is waiting for message");
7         dgs.receive(dgp); //receive message from client
8         System.out.println(new String(dgp.getData(), 0, dgp.getLength()));
9         String str = "Hello client, you are welcome";
10        byte [] bb = str.getBytes();
11        DatagramPacket dgp1 = new DatagramPacket(bb, bb.length,
12            dgp.getAddress(), dgp.getPort());
13        dgs.send(dgp1); //send message to client
14    }}
```

5 The URL Class

- The Java URL class represents URL.
 - ◆ URL is an acronym for Uniform Resource Locator.
 - ◆ It points to a resource on the World Wide Web.
 - ◆ A resource can be file, a directory, or it can be a reference object, such as a query to a database.
- Java's URL class has several constructors; each can throw a `MalformedURLException`.
 - ◆ `URL(String urlSpecifier)`
 - ◆ `URL(String protocolName, String hostName, int port, String path)`
 - ◆ `URL(String protocolName, String hostName, String path)`
 - ◆ `URL(URL urlObj, String urlSpecifier)`

5 The URL Class - Example

```
1 public static void main(String args[]) throws Exception{
2     //URL hp = new URL("file://localhost/home/yir/newJavaFile.java");
3     URL hp = new URL("http://www.google.com");
4     System.out.println("Protocol: " + hp.getProtocol());
5     System.out.println("Port: " + hp.getPort());
6     System.out.println("Host: " + hp.getHost());
7     System.out.println("File: " + hp.getFile());
8     BufferedReader in = new BufferedReader(new
        InputStreamReader(hp.openStream()));
9     String inputLine;
10    while ((inputLine = in.readLine()) != null)
11        System.out.println(inputLine);
12    in.close();
13 }
```

6 The URLConnection Class

- URLConnection is an abstract class that represents an active connection to a resource specified by a URL.
- in addition to the document itself in its raw form, you can access all the metadata specified by the protocol.
 - ◆ provides access to the HTTP header.
 - ◆ can configure the request parameters sent to the server.
 - ◆ can write data to the server as well as read data from the server.
 - ◆ Its `getOutputStream( )` method returns an output stream that can be used write to the resource.
- To instantiate an object of URLConnection class, use the `openConnection()` method of URL class

6 The URLConnection Class - Example

```
1 public static void main(String args[]) throws Exception{
2     //URL hp = new URL("http://www.google.com");
3     URL hp = new URL("file://localhost/home/yir/newJavaFile.java");
4     URLConnection hpCon = hp.openConnection();
5     long d = hpCon.getDate();
6     System.out.println("Date: " + new Date(d));
7     System.out.println("Content-Type: " + hpCon.getContentType());
8     d = hpCon.getExpiration();
9     System.out.println("Expires: " + new Date(d));
10    d = hpCon.getLastModified();
11    System.out.println("Last-Modified: " + new Date(d));
12    long len = hpCon.getContentLengthLong();
13    System.out.println("Content-Length: " + len);
```

6 The URLConnection Class - Example

```
14  int c;  
15  if(len != 0) {  
16      System.out.println("=== Content ===");  
17      InputStream input = hpCon.getInputStream();  
18      while (((c = input.read()) != -1)) {  
19          System.out.print((char) c);  
20      }  
21      input.close();  
22  } else {  
23      System.out.println("No content available.");  
24  }  
25 }
```

7 The HttpURLConnection Class

- The HttpURLConnection class is an abstract subclass of URLConnection
- It provides methods that are helpful when working specifically with http URLs.
 - ◆ It contains methods to get and set the request method,
 - ◆ Decide whether to follow redirects,
 - ◆ Get the response code and message,
 - ◆ And figure out whether a proxy server is being used.
- You obtain an HttpURLConnection by calling openConnection() on a URL object, then cast the result to HttpURLConnection.

7 The HttpURLConnection Class - Example

```
public static void main(String args[]) throws Exception{  
    URL hp = new URL("http://www.google.com");  
    HttpURLConnection hpCon = (HttpURLConnection) hp.openConnection();  
    System.out.println("Request method is " + hpCon.getRequestMethod());  
    System.out.println("Resp code is " + hpCon.getResponseCode());  
    System.out.println("Resp Message is " + hpCon.getResponseMessage());  
    Map<String, List<String>> hdrMap = hpCon.getHeaderFields();  
    Set<String> hdrField = hdrMap.keySet();  
    System.out.println("\nHere is the header:");  
    for(String k : hdrField) {  
        System.out.println("Key: " + k + " Value: " + hdrMap.get(k));  
    }  
}
```

8 HTTP Client API

- Has been added with jdk 11
- It provides enhanced, updated networking support for HTTP clients.
- Its three core elements are:
 - ◆ HttpClient:- Encapsulates an HTTP client.
 - ◆ HttpRequest:- Encapsulates a request.
 - ◆ HttpResponse:- Encapsulates a response.
- HttpClient is an abstract class, and instances are created via a factory method as:
 - ◆ `HttpClient myHC = HttpClient.newBuilder().build();`
 - ◆ `HttpClient.Builder` defines a number of methods that let you configure the builder.

8 HTTP Client API

```
HttpClient.Builder myBuilder = HttpClient.newBuilder()  
    .followRedirects(HttpClient.Redirect.NORMAL);  
HttpClient.myHC = myBuilder.build();
```

- Once you have an HttpClient instance, you can send a synchronous request by calling its send() method, as:

```
<T> HttpResponse <T> send(HttpRequest req,  
    HttpResponse .BodyHandler<T> handler) throws IOException,  
    InterruptedException
```

- req encapsulates the request and handler specifies how the response body is handled.

8 HTTP Client API

- To create an `HttpRequest` object, call `HttpRequest`'s `newBuilder()` method. Here are two of its forms:
 - ◆ `static HttpRequest.Builder newBuilder()`
 - ◆ `static HttpRequest.Builder newBuilder(URI uri)`
- `HttpRequest.Builder` lets you specify various aspects of the request, such as request method, header information.
- `HttpResponse` is an interface declared as `HttpResponse<T>`
 - ◆ `T` specifies the type of body, the body type is generic,
 - ➔ enables the body to be handled in a variety of ways.
- `HttpResponse` defines several methods that give you access to the information in the response.

8 HTTP Client API - Example

```
public class MyHttpClient {  
    public static void main(String[] args) throws Exception {  
        HttpClient myHC = HttpClient.newHttpClient();  
        HttpRequest myReq = HttpRequest.newBuilder(  
            new URI("http://www.google.com/")).build();  
        HttpResponse<InputStream> myResp = myHC.send(myReq,  
            HttpResponse.BodyHandlers.ofInputStream());  
        System.out.println("Response code is " + myResp.statusCode());  
        System.out.println("Request method is " + myReq.method());  
        HttpHeaders hdrs = myResp.headers();  
        Map<String, List<String>> hdrMap = hdrs.map();  
        Set<String> hdrField = hdrMap.keySet();  
    }  
}
```


8 HTTP Client API - Example

```
System.out.println("\nHere is the header:");
for(String k : hdrField)
    System.out.println("Key: " + k + " Value: " + hdrMap.get(k));
System.out.println("\nHere is the body: ");
InputStream input = myResp.body();
int c;
while((c = input.read()) != -1) {
    System.out.print((char) c);
} } }
```